

PRODUCT BACKLOG

Project: Dental Fable

Management System for a Pediatric Dental Clinic

Team: CodeXplorers | Version: 3.0 | Date: June 10th 2026

Technologies (Unit 1 — Frontend): PHP · MongoDB · Bootstrap 5 · JavaScript · Docker

Technologies (Unit 2 — Backend APIs): Node.js · TypeScript · Express · Prisma · PostgreSQL
(Supabase)

Team Members:

Andrés Cárdenas

Chuqui Kerly

Díaz Victoria

1. Project Description

Fábula Dental is a web-based clinical management system specializing in pediatric dentistry. The platform allows users to manage patients, track payments, and manage dental supply inventory, with differentiated access based on user role (Administrator, Dentist, Receptionist).

In Unit 2, the project evolved from a monolithic PHP/MongoDB architecture to a distributed two-server TypeScript REST API system, maintaining the original frontend as a legacy reference while building a production-grade backend.

Technology Stack — Unit 1 (Legacy Frontend, archived in old-frontend/)

- Backend: PHP 8.x with MVC architecture
- Database: MongoDB Atlas (collections: patients, payments, supplies)
- UI: Bootstrap 5.3, Vanilla JavaScript, HTML5, CSS3
- Infrastructure: Docker (Dockerfile)
- Authentication: PHP sessions with role control

Technology Stack — Unit 2 (Current Backend Architecture)

- CRUD Server (port 3000): Node.js + Express + TypeScript + Prisma ORM
- Business Logic Server (port 3001): Node.js + Express + TypeScript + fetch API
- Database: PostgreSQL via Supabase (tables: patients, payments, supplies)
- ORM: Prisma with pg adapter (PrismaPg + pgbouncer connection pooling)
- Deployment: GCP — CRUD: 34.28.34.201 | Business Logic: 34.122.231.144

Architecture Overview

The system uses two decoupled servers:

- CRUD Server: Handles all direct database operations (CREATE, READ, UPDATE, DELETE) using Prisma to query PostgreSQL.
- Business Logic Server: Consumes the CRUD server via HTTP fetch calls and applies domain calculations (payment status derivation, supply expiration classification, financial analysis) before returning enriched responses to clients.

System Actors

- Administrator: Manages the inventory of dental supplies
- Dentist: Registers and manages patient information
- Receptionist: Registers and manages patient payments
- Visitor: Consults the public information on treatments

2. User Stories

User stories describe the functionalities of the system from the perspective of each actor, following the standard agile format: As [role], I want [action], for [benefit].

ID	As...	Wanna...	For...	Acceptance Criteria
US-01	System user	Log in with username and password	access the module corresponding to my role	• Redirection based on role after successful login • Error message for invalid credentials • Fields cannot be empty
US-02	Dentist	Register a new patient with ID, name, DOB, and reason for consultation	have the patient's complete medical history in the system	• ID card with exactly 10 digits mandatory • Mandatory legal representative if patient < 18 years • Successful registration saves to PostgreSQL via CRUD API and returns 201
US-03	Dentist	View, edit, and delete patients from the list	keep patient information up-to-date	• Table loading patients from GET /fabuladental/patients • Online editing sends PUT to the API • Deletion sends DELETE and removes row without reloading
US-04	Receptionist	Record a payment indicating patient, amount, type and method	to keep track of the clinic's income and payments	• Payment type: Deposit or Final • Final → Completed; Deposit → Partial • Amount > 0, date not future, Patient ID = 10 digits • Method: Cash, Card, or Transfer
US-05	Receptionist	View the complete payment history with its current status	to have financial traceability	• All payments via GET /fabuladental/payments (BL) • Colors: Completed=green, Partial=yellow, Pending=red • Records can be edited and deleted
US-06	Administrator	Record dental supplies with name, quantity, cost, and dates	control the clinic's inventory	• Quantity min. 1 unit • Expiration date >= order date • Auto status: Current / NextExpiration / Expired
US-07	Administrator	View inventory with alerts for products nearing expiration	anticipate replenishment needs	• 'NextExpiration' if <= 30 days remaining • 'Expired' if date has passed • Differentiated visual indicator in table
US-08	Clinic visitor	View the list of available treatments without logging in	know about services before appointment	• Accessible from login header • Shows name, description, post-treatment recommendations • No authentication required
US-09	System	Retrieve payment history with calculated billing status	provide financial traceability per patient	• GET /fabuladental/payments → derived status list • GET /fabuladental/payments/:paymentId → specific record

ID	As...	Wanna...	For...	Acceptance Criteria
				• GET /fabuladental/patients/:patientId/payments → patient history
US-10	System	Perform financial analysis on inventory	support administrative decision-making	• GET /supplies/asset-valuations → total value • GET /supplies/restock-provisions → restock budget • GET /supplies/expiration-losses → loss from expired • GET /supplies/capital-risks → at-risk capital

3. Product Backlog

The Product Backlog below is ordered by epics and priority. Story points follow the Fibonacci scale (1, 2, 3, 5, 8).

High Priority | **Average Priority** | **Low Priority** | **Done** | **Pending**

ID	User Story	Description	Acceptance Criteria	Priority	Points	State
PB-01	Role-based login	As a system user, I want to log in with my credentials to access the features of my specific role.	Validates username/password. Redirects to correct view by role (Admin, Dentist, Receptionist). Error if credentials incorrect.	High	5 pts	Done
PB-02	Role-based access control	As the system administrator, I want each user to only have access to the modules that correspond to them.	Admin → Inventory only. Dentist → Patients only. Receptionist → Payments only.	High	3 pts	Done
PB-03	Log out	As an authenticated user, I want to securely log out to protect the clinic's information.	'Log Out' button destroys the PHP session and redirects to login.	High	2 pts	Done
PB-04	New patient registration	As a dentist, I want to register a new patient with their personal details.	Captures: ID (10 digits), full name, DOB, phone, reason, legal rep (if < 18). Stored via POST /fabuladental/patients.	High	8 pts	Done
PB-05	List of patients	As a dentist, I want to see all registered patients.	Table via GET /fabuladental/patients. Each record can be edited or deleted.	High	5 pts	Done
PB-06	Patient data editing	As a dentist, I want to edit an existing patient's data.	'Save' sends PUT /fabuladental/patients/:patientId. Changes reflected in real time.	High	5 pts	Done
PB-07	Patient removal	As a dentist, I want to delete a patient record.	DELETE /fabuladental/patients/:patientId. Record disappears without page reload.	Average	3 pts	Done
PB-08	Legal rep validation for minors	As a dentist, I want the system to require a legal representative for patients under 18.	Calculates age from DOB. Under 18 without guardian → registration fails with error.	High	3 pts	Done
PB-09	Payment record	As a receptionist, I want to record a patient payment with type, method, and amount.	Captures: Patient ID (10 digits), date, amount, type (Deposit/Final), method (Cash/Transfer/Card). Status auto-calculated. Stored via POST /fabuladental/payments.	High	8 pts	Done
PB-10	Payment history	As a receptionist, I want to see all recorded payments.	Shows payments with status (Pending/Partial/Completed) via GET /fabuladental/payments. Records editable and deletable.	High	5 pts	Done

ID	User Story	Description	Acceptance Criteria	Priority	Points	State
PB-11	Payment edition	As a receptionist, I want to edit a recorded payment.	Editable fields. Sends PUT /fabuladental/payments/:paymentId. Changes reflected immediately.	Average	3 pts	Done
PB-12	Visual indicator of payment status	As a receptionist, I want to see payment status with distinctive colors.	Pending = red/orange. Partial = yellow. Completed = green.	Average	2 pts	Pending
PB-13	Supply record	As an administrator, I want to register a new dental supply.	Captures: name, quantity (min. 1), unit cost, order date, expiration date. Stored via POST /fabuladental/supply.	High	8 pts	Done
PB-14	Inventory list with status	As an administrator, I want to see the supply inventory with expiration status.	Shows name, quantity, cost, dates. Status via POST /supplies/status-calculations: Current, NextExpiration ($\leq 30d$), Expired.	High	5 pts	Done
PB-15	Product expiration alert	As an administrator, I want automatic identification of supplies about to expire.	Calculates expiration diff. If ≤ 30 days $\rightarrow$ 'NextExpiration' with visual alert indicator.	High	3 pts	Done
PB-16	Editing and deleting supplies	As an administrator, I want to edit or delete supplies.	Edit: PUT /fabuladental/supplies/:id. Delete: DELETE /fabuladental/supplies/:id. List updates without reload.	Average	3 pts	Done
PB-17	Treatments page	As a clinic visitor, I want to see available treatments without logging in.	Accessible from login page. Shows table: name, description, post-treatment recommendations.	Average	3 pts	Done
PB-18	Patient search and filtering	As a dentist, I want to search for patients by name or ID.	Real-time search field filtering the table by name and ID number.	Average	5 pts	Pending
PB-19	Dashboard with statistics	As an administrator, I want summarized metrics on the dashboard.	Cards with dynamic counters. Data from API in real time.	Low	8 pts	Pending
PB-20	Secure database authentication	As the administrator, I want encrypted user credentials in the database.	Credentials stored in PostgreSQL with bcrypt hashing. Login queries DB.	High	8 pts	Pending
PB-21	Exporting reports	As a receptionist/admin, I want to export payment and inventory lists.	Export button in lists. Generates downloadable PDF/Excel file.	Low	8 pts	Pending
PB-22	TS migration — Payments CRUD API	As the dev team, we want to migrate the payments module to TypeScript/PostgreSQL.	paymentRoutes.ts in CRUD server. POST, PUT, DELETE, GET /fabuladental/payments. Validation: 10-digit ID, amount > 0, non-future date, valid type and method. Status	High	8 pts	Done

ID	User Story	Description	Acceptance Criteria	Priority	Points	State
			auto-calculated. Zero TypeScript errors.			Done
PB-23	TS migration — Payments Business Logic API	As the dev team, we want to expose payment business logic endpoints.	paymentsBusinessController.ts + paymentsBusinessRoutes.ts. GET /payments (history), GET /payments/:id (specific), GET /patients/:patientId/payments (patient history). Consumes CRUD via fetch. Zero TS errors.	High	5 pts	
PB-24	TS migration — Supplies Business Logic API	As the dev team, we want financial analysis endpoints for inventory.	suppliesBusinessController.ts: POST /status-calculations, GET /asset-valuations, GET /restock-provisions, GET /expiration-losses, GET /capital-risks. All deployed and functional.	High	8 pts	Done
PB-25	TS migration — Patients CRUD API	As the dev team, we want to migrate the patients module to TypeScript.	patientRoutes.ts in CRUD server. POST, GET, PUT, DELETE /fabuladental/patients. Validates 10-digit ID, legal rep for minors, birthday format. Prisma + BigInt IDs. Zero TS errors.	High	8 pts	Done

4. Sprint Planning

The following table summarizes the distribution of Backlog items in development Sprints, with the objective of each iteration and the current progress status.

Sprint	Items Included	Sprint Objective	Points	State
Sprint 1	PB-01, PB-02, PB-03, PB-04, PB-08, PB-17	Functional authentication, patient registration with validations, and a public treatment page.	19	Done
Sprint 2	PB-05, PB-06, PB-07, PB-09, PB-10, PB-13, PB-14, PB-15	Complete CRUD for patients, payment registration, inventory management with expiration alerts.	37	Done
Sprint 3	PB-11, PB-12, PB-16	Payment and inventory refinement: editing, deletion, and visual status indicators.	8	Done
Sprint 4 (Unit 2)	PB-22, PB-23, PB-24, PB-25	Migration of all modules (Patients, Payments, Supplies) from PHP/MongoDB to TypeScript/PostgreSQL. Two-server architecture: CRUD (port 3000) and Business Logic (port 3001). Deployed on GCP.	29	Done
Sprint 5	PB-18, PB-19, PB-20	Security improvements (DB-stored credentials), patient search, and statistical dashboard.	21	Planned
Sprint 6	PB-21	Export of reports in PDF/Excel for payments and inventory.	8	Planned

Total story points: 122 points | **Completed (Sprints 1–4):** 93 points | **Pending (Sprints 5–6):** 29 points

5. Technical and Non-Functional Requirements

5.1 Implemented Functional Requirements

- RF-01: Authentication system with differentiated roles (Admin, Dentist, Receptionist)
- RF-02: Complete CRUD of patients with Ecuadorian ID validation (10 digits) — TypeScript + Prisma
- RF-03: Validation of legal representative for patients under 18 years of age
- RF-04: Complete CRUD of payments with types (Deposit/Final) and methods (Cash/Transfer/Card) — TypeScript + Prisma
- RF-05: Automatic derivation of payment status (Deposit → Partial, Final → Completed) in Business Logic layer
- RF-06: Complete CRUD of supplies with automatic expiration status calculation — TypeScript + Prisma
- RF-07: Automatic alerts for products with an expiration date ≤ 30 days (NextExpiration status)
- RF-08: Public treatment page accessible without authentication
- RF-09: RESTful APIs for each module (GET, POST, PUT, DELETE) — TypeScript/Express
- RF-10: Business Logic endpoints for financial analysis: asset valuation, restock provisioning, expiration losses, capital risk
- RF-11: Payment history endpoint with calculated status per payment record
- RF-12: Patient payment history endpoint filtered by patient ID

5.2 Non-Functional Requirements

- RNF-01 Security: PHP session control by role (legacy frontend). Environment variables for DB credentials (.env, excluded from git).
- RNF-02 Usability: Responsive interface with Bootstrap 5. Compatible with mobile and desktop devices.
- RNF-03 Performance: CRUD operations do not require full page reload (fetch API). Business Logic communicates with CRUD via HTTP.
- RNF-04 Scalability: PostgreSQL (Supabase) with pgbouncer connection pooling for horizontal scaling.
- RNF-05 Portability: Deployable on GCP or any Node.js-compatible environment.
- RNF-06 Maintainability: Clean separation between CRUD server and Business Logic server. TypeScript strict mode enforced across both servers.
- RNF-07 Validation: Input validation in TypeScript (backend): 10-digit IDs, positive amounts, non-future dates, valid enum values.
- RNF-08 Type Safety: All new backend code compiled with TypeScript strict mode with zero type errors.

6. Definition of Done (DoD)

A backlog item is considered DONE when it meets all of the following criteria:

- The code is implemented and working in the development/production environment
- The acceptance criteria have been verified manually (Postman or browser)
- The functionality has been tested with real data in PostgreSQL (Supabase)
- There are no console errors or unhandled exceptions
- For TypeScript modules: compiles with zero errors using `tsc --noEmit`
- The code follows the project's architecture: routes/ and controllers/ under each server's src/
- The functionality has been reviewed and approved by the CodeXplorers team